RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institution
Affiliated to Anna University, Chennai,
Rajalakshmi Nagar, Thandalam – 602 105

BONAFIDE CERTIFICATE

Name: HARSHITHA. L.S

Academic Year: 2026-2027 **Semester:** V **Branch:** CSD

Register No. 2116 241701019

Certified that this is the bonafide record of work done by the above student in the IT23E31 - Graphics & Multi Media *Laboratory during the academic year* 2026-2027

Signature of Faculty in-charge

Submitted for the Practical Examination held on.....

Internal Examiner **External Examiner**

HOME PAGE

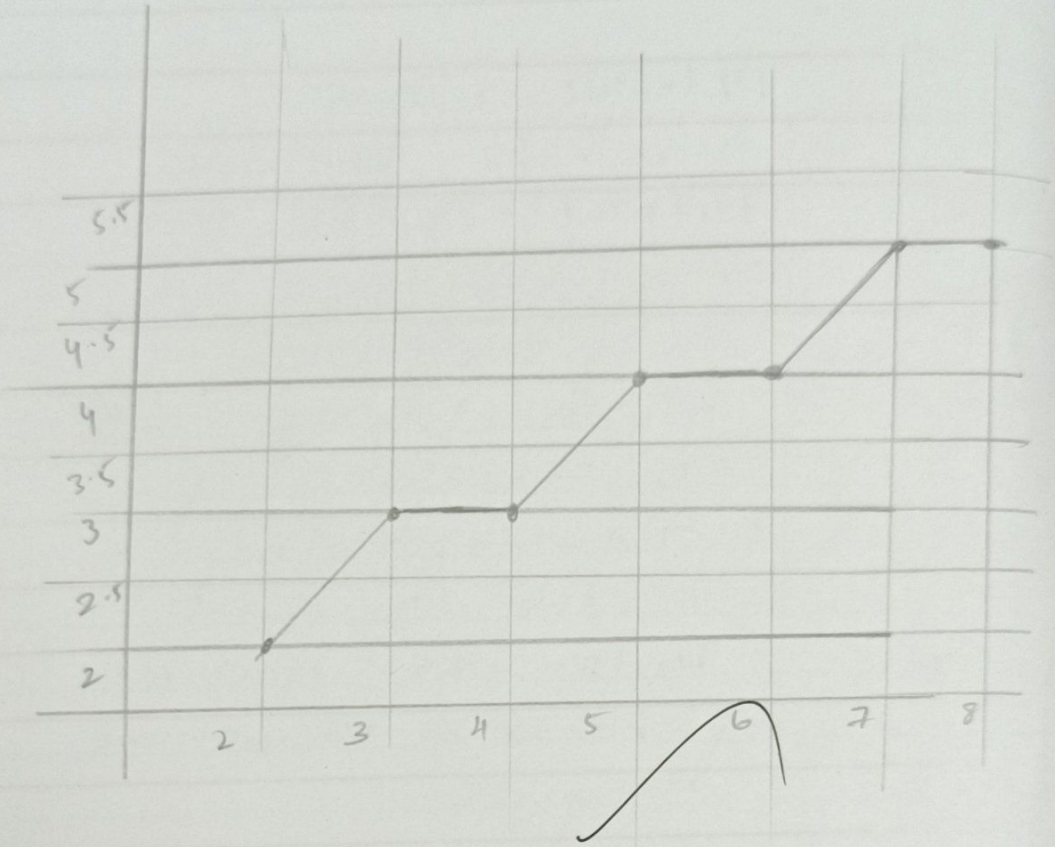
M Harshitha.L.S CLASS / SEC. SUB.

S. No.	Date	Description	Page No.
1)		Bresenhams Line Algorithm	
2)		Bresenhams Circle Algorithm	
3)		Midpoint Ellipse	
4)		Bresenhams Rectangle	
5)		Midpoint - Olympic symbol	
6)		Scan conversion of a polygon	
7)		flood fill algorithm	
8)		polygon clipping	
9)		2D transformation	
10)		3D transformation	
11)		Render 3D object	
12)		Multimedia application	
13)		Webcam or Microphone	
14)		Messenger	
15)		Blender.	

Completed

20/12/25

Bresenham's Line Drawing Algorithm



4

Bresenhams Line Drawing Algorithm

Aim:

To draw a line using Bresenham's Algorithm.

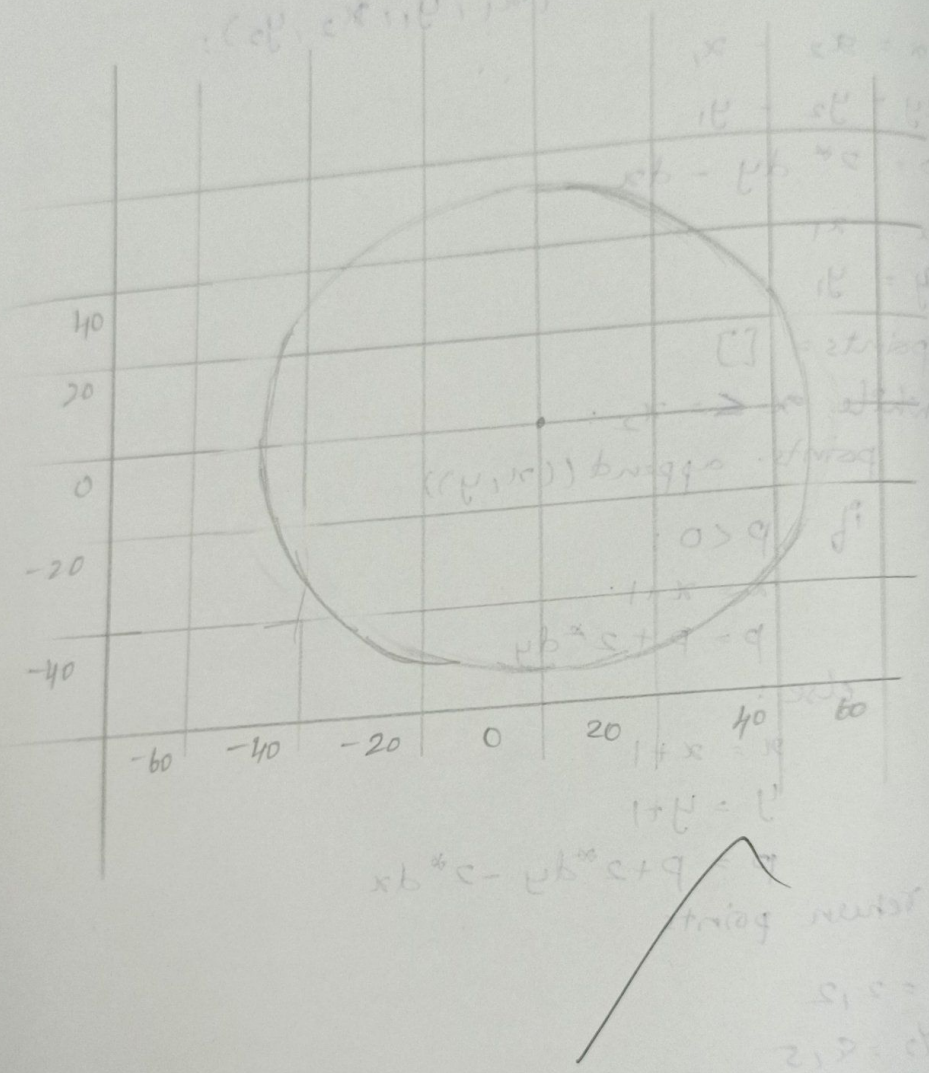
Program:

```
import matplotlib.pyplot as plt
def bresenhams_line(x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1
    p = 2*dy - dx
    x = x1
    y = y1
    points = []
    while x <= x2:
        points.append((x, y))
        if p < 0:
            x = x + 1
            p = p + 2*dy
        else:
            x = x + 1
            y = y + 1
            p = p + 2*dy - 2*dx
    return points
x1, y1 = 2, 2
x2, y2 = 9, 5
points = bresenhams_line(x1, y1, x2, y2)
x = [point[0] for point in points]
y = [point[1] for point in points]
plt.plot(x, y, 'o-', markersize = 8)
plt.grid(True)
plt.xlabel("X") plt.ylabel("Y")
plt.title("Bresenhams Line Drawing Algorithm") plt.sh
```

Result:

Thus, a line using Bresenhams Algorithm has been completed.

Bresenham's Circle



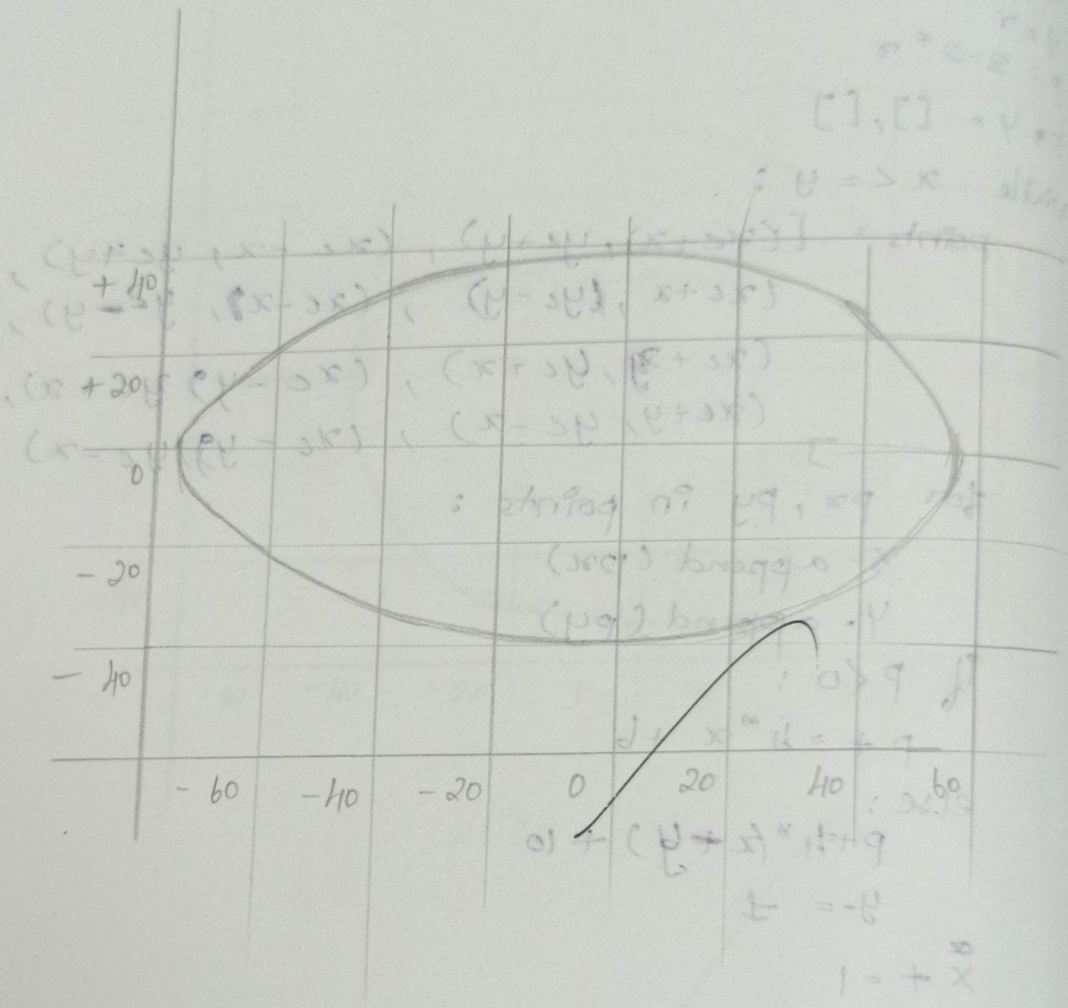
Bresenhams Circle Algorithm

Aim:
To draw a circle using Bresenhams Algorithm.

Program:

```
import matplotlib.pyplot as plt
def circle(xc, yc, r):
    x = 0
    y = r
    p = 3-2*r
    x, y = [], []
    while x <= y:
        points = [(xc+x, yc+y), (xc-x, yc+y),
                  (xc+x, yc-y), (xc-x, yc-y),
                  (xc+y, yc+x), (xc-y, yc+x),
                  (xc+y, yc-x), (xc-y, yc-x)
                 ]
        for px, py in points:
            x.append(px)
            y.append(py)
        if p<0:
            p += 4*x +6
        else:
            p+=4*(x+y)+10
            y-= 1
        x+=1
    plt.scatter(x, y, s=20)
    plt.axes('equal')
    plt.grid()
    plt.title("Bresenhams circle")
    plt.show()
circle(0, 0, 50)
```

Result:
Thus, a circle using bresenhams algorithm has been completed successfully.


+40
+20
0
-20
-40
-60
-40
-20
0
20
40
60

Bresenhams Ellipse Algorithm
Midpoint

Aim:

To draw an ellipse using Bresenhams Ellipse drawing Algorithm. (Midpoint)

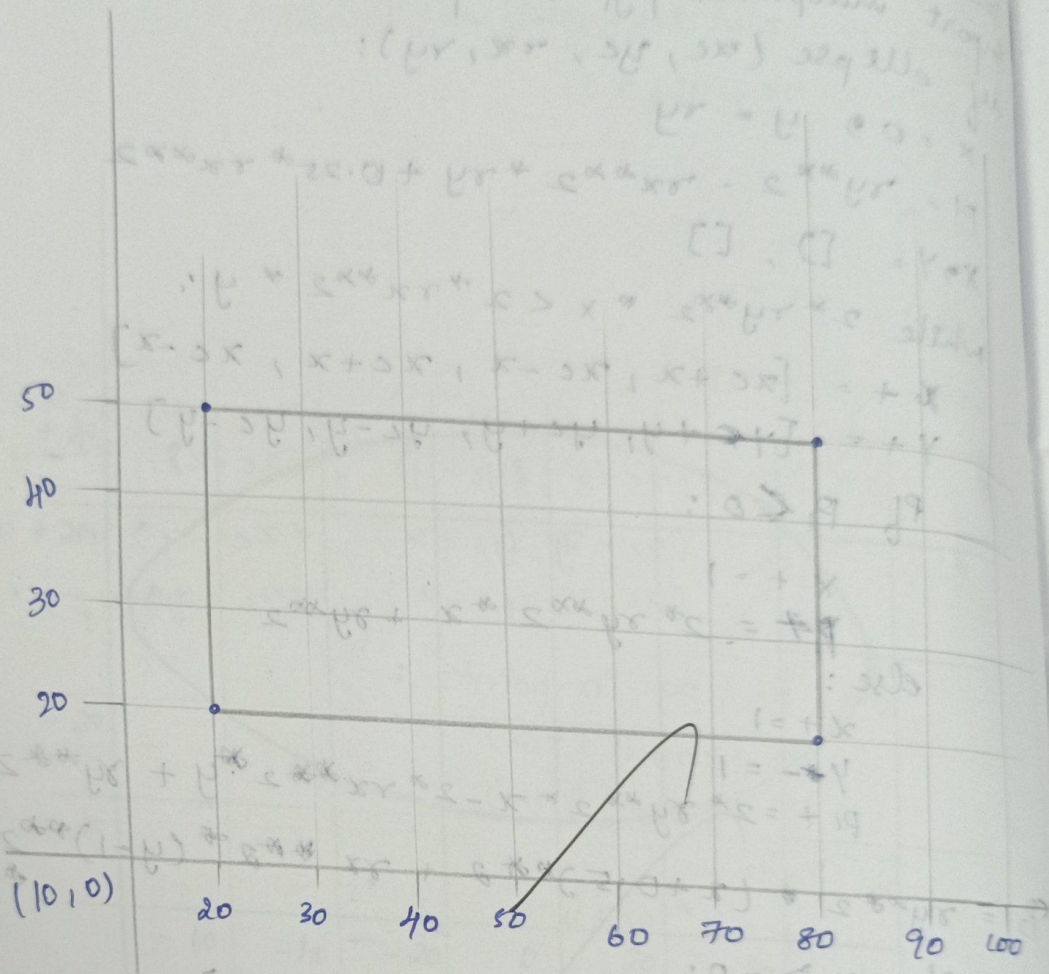
Program:

```
import matplotlib.pyplot as plt
def ellipse(xc, yc, rx, ry):
    x = 0; y = ry
    p1 = ry**2 - rx**2 * ry + 0.25 * rx**2
    X, Y = [], []
    while 2*ry**2 * x < 2*rx**2 * y:
        X += [xc+x, xc-x, xc+x, xc-x]
        Y += [yc+y, yc+y, yc-y, yc-y]
        if p1 < 0:
            x += 1
            p1 += 2*ry**2 * x + ry**2
        else:
            x += 1
            y -= 1
            p1 += 2*ry**2 * x - 2*rx**2 * y + ry**2
    p2 = ry**2 * (x+0.5)**2 + rx**2 * (y-1)**2 - rx**2 * ry**2
    while y >= 0:
        X += [xc+x, xc-x, xc+x, xc-x]
        Y += [yc+y, yc+y, yc-y, yc-y]
        if p2 > 0:
            y -= 1; p2 += rx**2 - 2*rx**2 * y
        else:
            x += 1; y -= 1; p2 += 2*ry**2 * x - 2*rx**2 * y + rx**2
    plt.scatter(X, Y, s=20)
    plt.axis("equal")
    plt.grid()
    plt.title("Mid point Ellipse")
    plt.show()
```

Result:

Thus, an ellipse was drawn using midpoint Algorithm successfully.

Bresenham Rectangle



Bresenhams Rectangle

Aim:

To draw a rectangle using Bresenhams Algorithm.

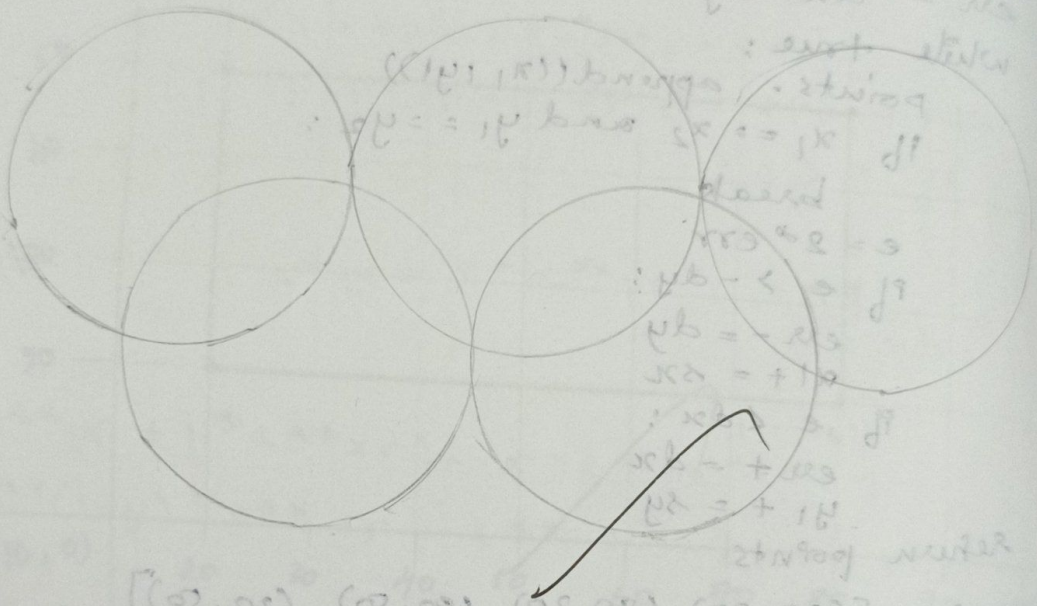
Program:

```
import matplotlib.pyplot as plt
def bresenham(x1, y1, x2, y2):
    points = []
    dx = abs(x2 - x1)
    dy = abs(y2 - y1)
    sx = 1 if x1 < x2 else -1
    sy = 1 if y1 < y2 else -1
    err = dx - dy.
    while true:
        points. append((x1, y1))
        if x1 == x2 and y1 == y2:
            break
        e = 2* err
        if e > -dy:
            err -= dy
            x1 += sx
        if e < dx:
            err += dx
            y1 += sy
    return points
corners = [(20,20), (80,20), (80,50), (20,50)]
X, Y = [], []
for i in range (4):
    p = bresenham(*corners[i], *corners [(i+1)%4])
    for x, y in p:
        x. append (x)
        y. append (y)
plt. scatter (X, Y, s = 15)
plt. axis ("equal") plt-grid() plt. title ("Bresenham's Rectangle")
plt. show()
```

Result:

Thus a rectangle was drawn using bresenhams algorithm.

Olympic symbol using Midpoint Circle Algorithm



Olympic symbol - Midpoint

Aim:

To draw the olympic rings using Midpoint Algorithm

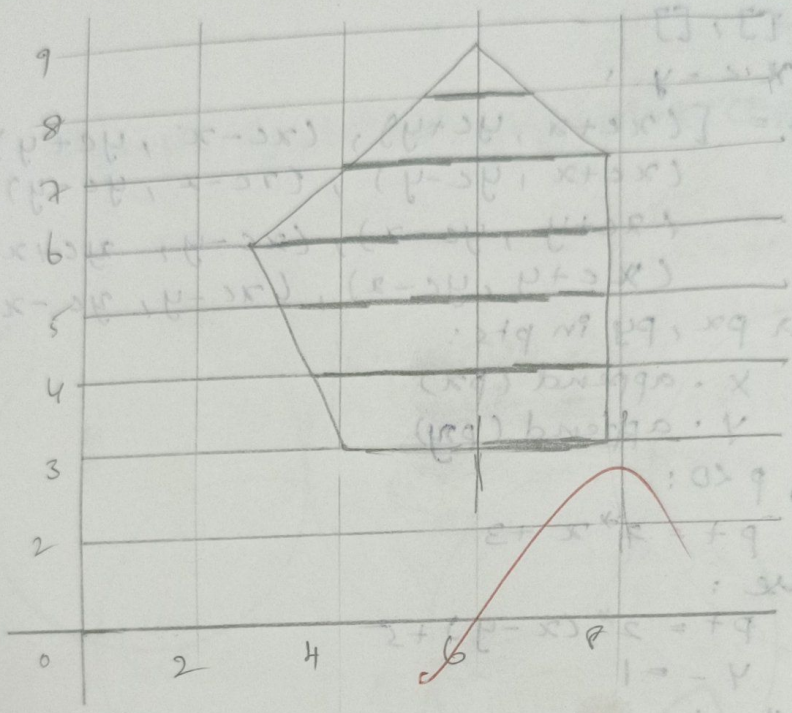
Program:

```
import matplotlib.pyplot as plt
def midpoint_circle(xc,yc,r):
    x,y,P = 0,r,1-r
    X,Y = [],[]
    while x<=y:
        pts = [(xc+x,yc+y),(xc-x,yc+y),
               (xc+x,yc-y),(xc-x,yc-y),
               (xc+y,yc-x),(xc-y,yc+x),
               (xc+y,yc-x),(xc-y,yc-x)]
        for px,py in pts:
            X.append(px)
            Y.append(py)
        if P<0:
            P+=2*x+3
        else:
            P+=2*(x-y)+5
            y-=1
        x+=1
    return X,Y
rings = [(40,60),(80,60),(120,60),(60,35),(100,3
plt.figure(figsize=(8,5))
for xc,yc in rings:
    x,y = midpoint_circle(xc,yc,20)
    plt.scatter(x,y,s=4)
plt.axis("equal")
plt.axis("off")
plt.title("Olympic Symbol using Midpoint Circle
                    Algorithm")
plt.show()
```

Result:

Thus, the Olympic ring symbol was drawn successfully using midpoint circle Algorithm.

Scan-line polygon fill



Scan Conversion of a polygon

Aim:

Write a program to fill a polygon using the scan-line fill algorithm.

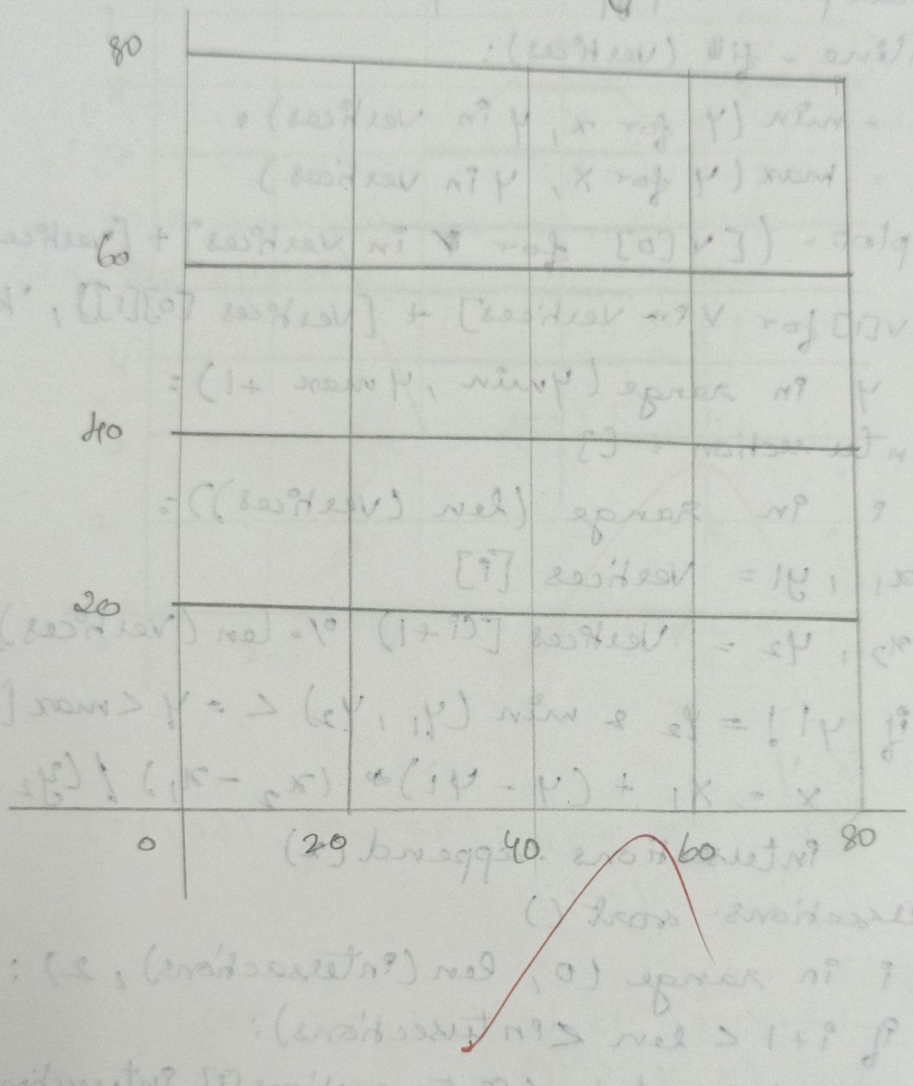
Program:

```
import matplotlib.pyplot as plt
def scanline_fill(vertices):
    ymin = min(y for x, y in vertices)
    ymax = max(y for x, y in vertices)
    plt.plot([v[0] for v in vertices] + [vertices[0][0]],
             [v[1] for v in vertices] + [vertices[0][1]], 'k-')
    for y in range(ymin, ymax+1):
        intersection = []
        for i in range(len(vertices)):
            x1, y1 = vertices[i]
            x2, y2 = vertices[(i+1) % len(vertices)]
            if y1 != y2 and min(y1, y2) <= y < max(y1, y2):
                x = x1 + (y - y1) * (x2 - x1) / (y2 - y1)
                intersections.append(x)
        intersections.sort()
        for i in range(0, len(intersections), 2):
            if i+1 < len(intersections):
                plt.plot([intersections[i], intersections[i+1]],
                         [y, y], 'r-')
vertices = [(2,2), (8,2), (7,7), (4,9), (1,6)]
scanline_fill(vertices)
plt.grid()
plt.axis('equal')
plt.title('Scan-line Polygon fill')
plt.show()
```

Result:

Thus the polygon filling algorithm was executed successfully.

flood fill algorithm



flood fill algorithm

Aim:
To write a program to fills a closed polygon from a seed point using 4-connected flood fill.

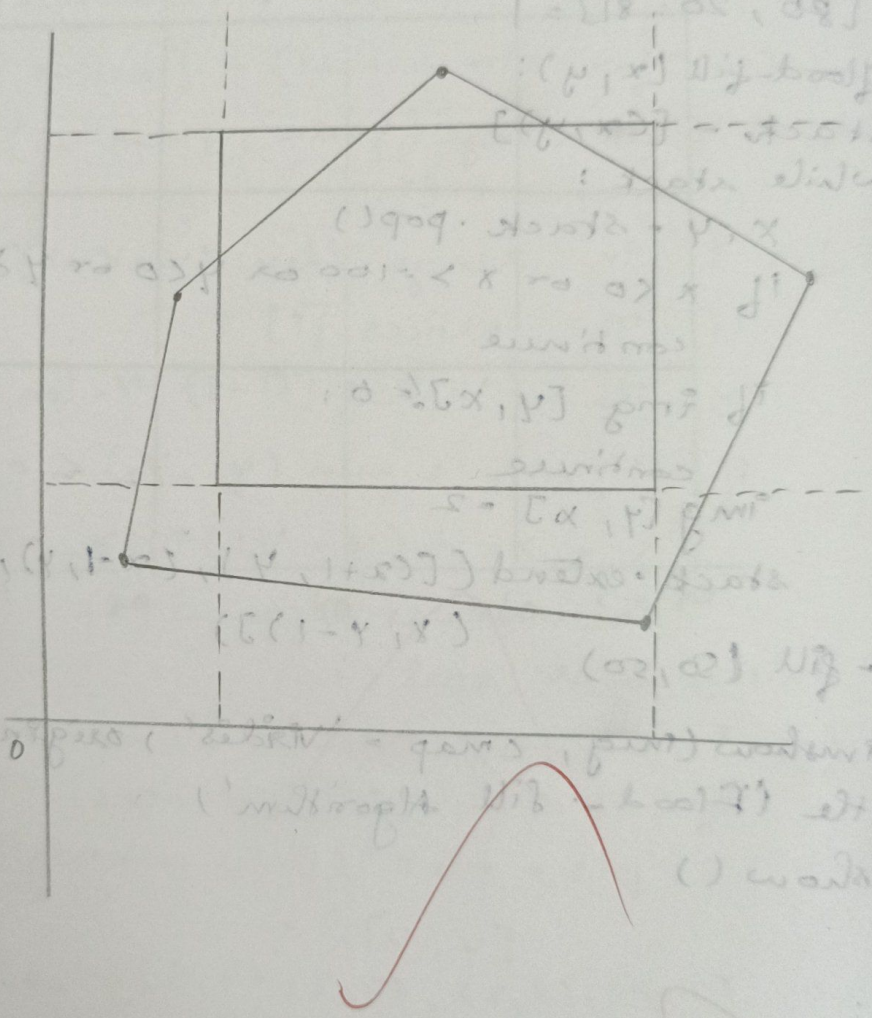
Program:

```
import matplotlib.pyplot as plt
import numpy as np
img = np.zeros((100,100), dtype = int)
img[20:81, 20] = 1
img[20:81, 80] = 1
img[20, 20:81] = 1
img[80, 20:81] = 1
def flood_fill(x, y):
    stack = [(x, y)]
    while stack:
        x, y = stack.pop()
        if x<0 or x>=100 or y<0 or y>=100:
            continue
        if img[y, x] != 0:
            continue
        img[y, x] = 2
        stack.extend([(x+1, y), (x-1, y), (x, y+1),
                      (x, y-1)])
flood_fill(50,50)
plt.imshow(img, cmap = 'viridis', origin = 'lower')
plt.title('Flood-fill Algorithm')
plt.show()
```

Result:

Thus, a program to fill closed polygons is executed successfully.

Sutherland-Hodgman polygon clipping



Polygon clipping

Aim:
To write a code to clip the polygon to a specified window & displays the clipped polygon.

Program:

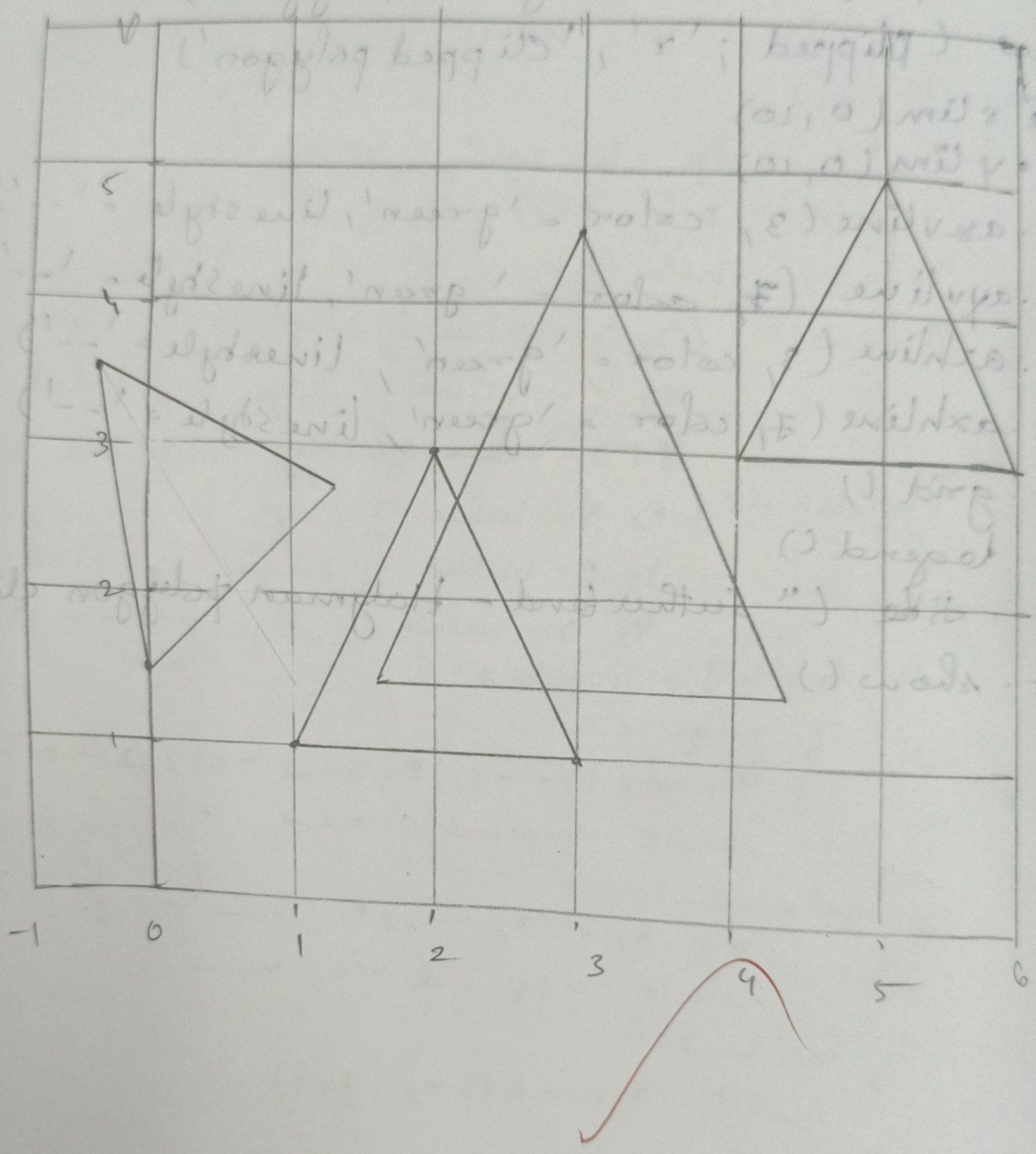
```
import matplotlib.pyplot as plt
def clip(poly, edge, value):
    result = []
    for i in range(len(poly)):
        x1, y1 = poly[i-1]
        x2, y2 = poly[i]
        inside1 = (x1 >= value if edge == 0 else
                   x1 <= value if edge == 1 else
                   y1 >= value if edge == 2 else y1 <= value
        inside2 = (x2 >= value if edge == 0 else
                   x2 <= value if edge == 1 else
                   y2 >= value if edge == 2 else y2 <= value
        if inside1 != inside2:
            if edge < 2:
                y = y1 + (y2 - y1) * (value - x1) / (x2 - x1)
                result.append((value, y))
            else:
                x = x1 + (x2 - x1) * (value - y1) / (y2 - y1)
                result.append((x, value))
        if inside2:
            result.append((x2, y2))
    return result
def polygon_clip(poly, xmin, ymin, xmax, ymax):
    poly = clip(poly, 0, xmin)
    poly = clip(poly, 1, xmax)
    poly = clip(poly, 2, ymin)
    poly = clip(poly, 3, ymax)
    return poly
poly = [(1,2), (6,1), (9,6), (5,9), (2,7)]
clipped = polygon_clip(poly, 3, 3, 7, 7)
```

```
def draw(poly, color, label):
    if poly:
        p = poly + [poly[0]]
        plt.plot([x for x, y in p],
                 [y for x, y in p], color=color, label=label)

draw(poly, 'b', 'Original Polygon')
draw(clipped, 'r', 'Clipped polygon')
plt.xlim(0,10)
plt.ylim(0,10)
plt.axvline(3, color='green', linestyle='--')
plt.axvline(7, color='green', linestyle='--')
plt.axhline(3, color='green', linestyle='--')
plt.axhline(7, color='green', linestyle='--')
plt.grid()
plt.legend()
plt.title("Sutherland-Hodgman polygon clipping")
plt.show()
```

Result:

Thus the code to clipped a polygon in a specific window is executed successfully.


5
4
3
2
1
-1
0
1
2
3
4
5
6

Basic 2D Transformations

Aim:
To write a code to perform basic transformations on 2D objects (shapes).

Program:

```
import matplotlib.pyplot as plt
import numpy as np
T = np.array([[1,1],[3,1],[2,3],[1,1]])

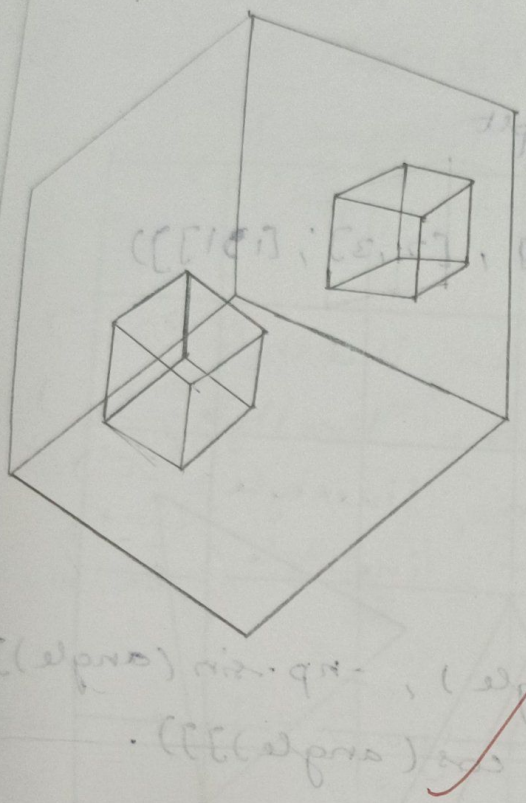
Tx, Ty = 4,2
translated = T + [Tx, Ty]

sx, sy = 1.5, 1.5
scaled = T * [sx, sy]

angle = np.radians(45)
R = np.array([[np.cos(angle), -np.sin(angle)],
              [np.sin(angle), np.cos(angle)]])
rotated = T @ R.T
plt.plot(T[:,0], T[:,1], label='Original')
plt.plot(translated[:,0], translated[:,1], lable='translation')
plt.plot(scaled[:,0], scaled[:,1], lable='scaling')
plt.plot(rotated[:,0], rotated[:,1], lable='rotation')
plt.axis("equal")
plt.grid()
plt.legend()
plt.show()
```

Result:
Thus, a program to perform basic 2D transformations was executed successfully.

3D transformation



Basic 3D transformations

Aim:

To write a code to perform Basic 3D transformations.

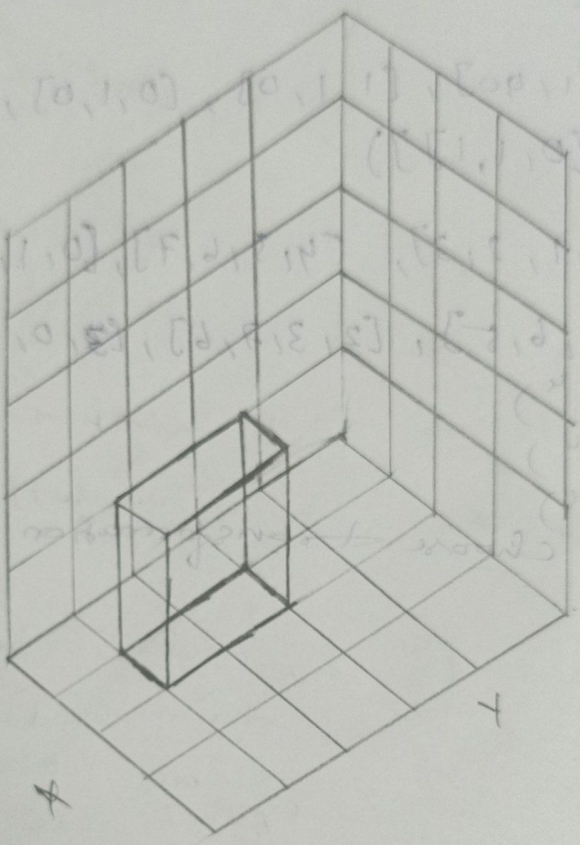
Program:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
cube = np.array([
    [0,0,0], [1,0,0], [1,1,0], [0,1,0], [0,0,1], [1,0,
    [1,1,1], [0,1,1]])
cube_faces = [[0,1,2,3], [4,5,6,7], [0,1,5,4],
              [1,2,6,5], [2,3,7,6], [3,0,4,7]]
print("1.Translate")
print("2. Scaling")
print("3. Rotation")
ch = int(input("Choose transformation:"))
p = cube.copy()
if ch==1:
    p += [2,1,1]
if ch==2:
    p *= 1.5
if ch==3:
    a = np.radians(45)
    R = np.array([[np.cos(a), -np.sin(a), 0], [np.sin(a), np.
                  [0,0,1]])  P=P@R.T
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("z")  ax.set_title("3D cube transformati
plt.show()
```

Result:

Thus, a code to perform basic 3D transformation was executed successfully.

3D objects with color & shading



Render 3D objects

Aim:
To create & render 3D objects using polygons and apply basic color & shading techniques.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
fig = plt.figure()
ax = fig.add_subplot(111, projects='3d')
v = np.array([[0,0,0], [1,0,0], [1,1,0], [0,1,0], [0,0,1],
              [1,0,1], [1,1,1], [0,1,1]])
faces = [[0,1,2,3], [4,5,6,7], [0,1,5,4], [1,2,6,5],
         [2,3,7,6], [3,0,4,7]]
ax.add_collection3d(Poly3DCollection(
    [v[f] for f in faces], facecolors=['red', 'blue',
    'green', 'orange', 'cyan', 'yellow'],
    edgecolors='black', shade=True, alpha=0.9))

ax.set_xlim(-1, 4)
ax.set_ylim(-1, 4)
ax.set_zlim(0, 3)
ax.set_xlable("x")
ax.set_ylable("y")
ax.set_zlable("z")
ax.set_title("3D objects with color & shading")
plt.show()
```

Result:
Thus, a program to create & render 3D objects has been executed successfully.

Multi Media Application

Aim:

To create a multimedia Application that integrates Images, sound and video in a simple User Interface

Program:

```
import tkinter as tk
from tkinter import filedialog
from PIL import Image, ImageTk
from playsound import playsound
import cv2
import threading
def image():
    file = filedialog.askopenfilename()
    if file:
        image = Image.open(file)
        img.thumbnail((300,300))
        pic = ImageTk.PhotoImage(img)
        panel.config(image=pic)
        panel.image = pic
def audio():
    file = filedialog.askopenfilename()
    if file:
        threading.Thread(
            target=playsound, args=(file,), daemon=True
        ).start()
def video():
    file = filedialog.askopenfilename()
    if file:
        cap = cv2.VideoCapture(file)
        while cap.isOpened():
            ret, frame = cap.read()
            if not ret:
                break
            cv2.imshow("Video player", frame)
            if cv2.waitKey(25) & 0xFF == ord('q'):
                break
        cap.release()
        cv2.destroyAllWindows()
```

```
root = tk.TK()
root.title("Multi-media Application")
root.geometry("400x450")
tk.Label(root, text="Multimedia Player",
        font=("Arial", 16, "bold")).pack(pady=10)

panel = tk.Label(root)
panel.pack()
tk.Button(root, text="Show Image", command=image).
                pack(pady=5)
tk.Button(root, text="Show Audio", command=Audio).
                pack(pady=5)
tk.Button(root, text="Show Video", command=Video).
                pack(pady=5)

root.mainloop()
```

Result:

Thus, the program to create a multimedia applications that integrates Image, Audio & Video was executed successfully.

Webcam or Microphone

Aim:

To capture Video/Audio from Webcam or Microphone and display on Multimedia Interface

Program:

```
import cv2
import sounddevice as sd
import tkinter as tk
from PIL import Image, ImageTk
def video():
    cap = cv2.VideoCapture(0)
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        cv2.imshow("Webcam", frame)
        if cv2.waitKey(1) == ord('q'):
            break
    cap.release()
    cv2.destroyAllWindows()
def audio():
    sound = sd.rec(16000*5, samplerate = 16000, channel
                                                =1)
    sd.wait()
    print("Audio Captured!")
root = tk.Tk()
root.title("Multimedia Interface")
tk.Button(root, text="Capture Video", command=video).
                pack(pady = 10)
tk.Button(root, text="
                capture Audio", Command=Audi
                pack(pady=10)
root.mainloop()
```

Result:

Thus, the capture of Video / Audio was done successfully.

Messenger

Aim:
To create an Application to send and receive Multimedia Messages (Text, Image, Audio).

Program:

```
import tkinter as tk
from tkinter import filedialog
from PIL import Image, ImageTk
from playsound import playsound
def send():
    msg = entry.get()
    chat.insert(tk.END, "You: " + msg + "\n")
def image():
    f = filedialog.askopenfilename()
    if f:
        img = ImageTk.PhotoImage(Image.open(f).resize((150, 150)))
        panel.config(image=img)
        panel.image = img
        chat.insert(tk.END, "You: " + "Image Received\n")
def audio():
    f = filedialog.askopenfilename()
    if f:
        playsound(f)
root = tk.Tk()
root.title("Multimedia Messenger")
entry = tk.Entry(root, width=30)
entry.pack()
tk.Button(root, text="Send Text", command=send).pack()
tk.Button(root, text="Send Image", command=image).pack()
tk.Button(root, text="Send Audio", command=audio).pack()
chat = tk.Text(root, height=8, width=35)
chat.pack()
panel = tk.Label(root)
panel.pack()
root.mainloop()
```

Result:
Thus, an application was created successfully.

Blender

Aim:
To create a simple 3D Model in Blender and Apply basic shading and textures.

Procedure:

1) Open Blender and add basic mesh object using Add > Mesh.
2) Model a house or object using Edit mode & Extrusion
3) Apply Materials by adjusting colour, roughness & specular values.
4) Add Image or procedural textures using the shader Editor and UV mapping.
5) Add lights & position the Camera.
6) Select Eevee or Cycles, adjust render settings, and click Render Image.

Result:

A 3D model was created, textured, shaded, and rendered successfully in Blender.